

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO
CIÊNCIA/ENGENHARIA DE COMPUTAÇÃO

DAVID MAQUARTE GUMES E SILVA
DIMITRY DEVEZA
DJAN CARLOS DE MOURA FERREIRA JUNIOR
EDUARDO VITOR OGAWA
ENRICO MASSARIOL CALIARI
ENRICO POLEZ FERREIRA PINTO
ENZO VENTURINI BAPTISTA
GABRIEL CARMINATI MIRANDA
GEORGE CIRILO DO NASCIMENTO
GUIDO AGNEZI DENIZ
GUILHERME CARDOSO DELATORRE

RCA 1802
Arquitetura e Organização de Computadores

DAVID MAQUARTE GUMES E SILVA
DIMITRY DEVEZA
DJAN CARLOS DE MOURA FERREIRA JUNIOR
EDUARDO VITOR OGAWA
ENRICO MASSARIOL CALIARI
ENRICO POLEZ FERREIRA PINTO
ENZO VENTURINI BAPTISTA
GABRIEL CARMINATI MIRANDA
GEORGE CIRILO DO NASCIMENTO
GUIDO AGNEZI DENIZ
GUILHERME CARDOSO DELATORRE

RCA 1802
Arquitetura e Organização de Computadores

Relatório acadêmico do trabalho opcional da disciplina de Arquitetura e Organização de Computadores, referente à projeção de um modelo de processador RCA 1802.

Professor: Jadir Eduardo Souza Lucas.

SUMÁRIO

1 INTRODUÇÃO	3
2 INSTRUÇÕES	4
3 ULA	4
4 REGISTRADORES	6
5 UNIDADE DE CONTROLE E μROM	8
6 PLA / DECODIFICADOR	9
7. MÓDULO DE MEMÓRIA E INTERFACE DE I/O	10
7.1 Barramento Multiplexado	10
7.2 Interface de Memória	10
7.3 Controlador DMA (Direct Memory Access)	11
7.4 Interface EF e Q	12
7.4.1 Flags Externas (EF1 a EF4)	12
7.4.2 Saída Programável (Q)	12
7.5 Diagrama do Módulo	12
8. TESTES EM ASSEMBLY	13
8.1 Visão Geral dos Programas de Teste	13
8.2 test_regs	13
8.3 test_alu_arith	14
8.4 test_alu_logic	15
8.5 test_shift	15
8.6 test_load_store	15
8.7 test_branch	16
8.8 test_io	16
8.9 test_cypmem	17
9 RESPONSABILIDADES	18
REFERÊNCIAS	18

1 INTRODUÇÃO

O estudo de arquiteturas de processadores constitui um dos pilares da formação em Ciência da Computação e Engenharia, permitindo compreender os mecanismos que governam o funcionamento de sistemas computacionais desde sua camada mais básica. Nesse contexto, processadores históricos apresentam valor pedagógico singular: ao reduzir a complexidade dos sistemas modernos a conjuntos de instruções mais acessíveis, tornam visíveis princípios de organização e controle que permanecem presentes nas arquiteturas contemporâneas.

O presente relatório descreve o desenvolvimento de um modelo funcional do microprocessador RCA CDP 1802, implementado no simulador Logisim Evolution. Lançado pela Radio Corporation of America em 1974, o CDP 1802 — conhecido como COSMAC (*Complementary Symmetry Monolithic Array Computer*) — foi o primeiro microprocessador fabricado em tecnologia CMOS, o que lhe conferia consumo reduzido, operação em ampla faixa de tensão e a possibilidade de suspender o clock sem perda de estado interno. Essas características tornaram-no predominante em aplicações embarcadas e aeroespaciais, com registros de uso em missões da NASA como a sonda Galileu e o Telescópio Espacial Hubble.

O projeto foi desenvolvido por um grupo de onze alunos, organizados em sete frentes de trabalho: documentação do conjunto de instruções, Unidade Lógico-Aritmética, banco de registradores, unidade de controle e μ ROM, decodificador de instruções por PLAs, interface de memória com barramento multiplexado e controlador DMA, e programas de teste em assembly. A composição deste relatório foi realizada coletivamente, tendo o datasheet oficial da Intersil Corporation como referência técnica primária.

As seções seguintes detalham cada módulo implementado, as decisões de projeto adotadas e as responsabilidades individuais de cada integrante.

2 INSTRUÇÕES

O conjunto de instruções do CDP1802 é composto por 91 instruções distribuídas em oito categorias funcionais: carga e armazenamento, operações em registradores, aritmética, lógica, deslocamentos, desvios curtos, desvios longos e controle/entrada-saída. A documentação completa foi organizada em uma planilha compartilhada com 14 campos por instrução: categoria, opcode em hexadecimal, representação binária com separação dos campos I e N, mnemônico, operandos, número de bytes, ciclos de máquina, ciclos de clock, flags afetadas, modo de endereçamento, operação na notação formal do datasheet e observações.

A principal referência utilizada foi o datasheet oficial da Intersil Corporation para o CDP1802 (INTERSIL CORPORATION, 1999), especificamente a Tabela 1 (págs. 3-23 a 3-26), que lista todas as instruções com suas operações formais, e a Tabela 2 (págs. 3-27 a 3-29), que detalha o comportamento do barramento durante cada ciclo de execução.

A codificação dos opcodes segue um padrão uniforme: o nibble superior do byte de instrução (campo I, bits 7–4) determina a classe da operação, enquanto o nibble inferior (campo N, bits 3–0) seleciona o registrador operando entre R0 e R15. Instruções com opcode fixo, como LDI (0xF8) e STXD (0x73), utilizam o campo N como parte do próprio código de operação. As instruções de um byte consomem dois ciclos de máquina (16 ciclos de clock), com exceção dos desvios longos, skips longos e NOP, que requerem três ciclos de máquina (24 ciclos de clock). O único flag de uso geral é o DF (*Data Flag*), afetado exclusivamente por instruções aritméticas e de deslocamento — as instruções lógicas não o alteram.

3 ULA

A Unidade de Lógica e Aritmética do RCA 1802 suporta as operações lógicas básicas de AND, OR e XOR, e operações aritméticas básicas de adição e subtração. Ela recebe como operandos *Bus In* e *D* de 8 bits, o qual *Bus In* é a entrada que recebe os dados advindos do barramento principal de dados do *Datapath*. Além disso, a entrada *D* vem de um registrador especial de mesmo nome que fica

conectado diretamente na saída da ULA, ou seja, esse registrador é, ao mesmo tempo, entrada e saída da Unidade Lógica Aritmética. Por causa disso, é necessária a existência de uma instrução de COPY, a qual recebe o dado do barramento e grava diretamente no registrador D, podendo assim, ser possível operações com 2 operandos diferentes.

Com isso, foram feitas as 8 operações da ULA desenhadas no processador original: COPY, OR, AND, XOR, ADD, SUB, SHIFT e SUB. As operações são selecionadas respectivamente por meio de uma entrada de 3 bits denominada *ALU OP*. A operação de SUB aparece 2 vezes pois, no desenho do RCA 1802, as operações $A - B$ e $B - A$ foram divididas em *ALU OPs* diferentes, sendo a operação $A - B$ com *ALU Op* igual a 5 e $B - A$ igual a 7 - considerando A como a entrada do barramento e B a entrada do registrador D. A operação de SHIFT é, na verdade, uma operação dupla, sendo necessário a existência de um bit adicional na entrada *Shifter Direction* o qual diz para que lado será realizado o deslocamento. Além disso, todo deslocamento será feito utilizando do mecanismo de *Carry In*, que é inserido no bit mais à direita em um deslocamento a esquerda e mais a esquerda em um deslocamento a direita. Ademais, blocos operacionais separados foram criados e integrados para a realização das operações de deslocamento à esquerda e direita com *carry*.

A saída principal da ULA, denominada *Alu Out* no seu circuito, são 8 bits que são escritos para o registrador D no banco de registradores. Ao invés de um conjunto de bits representando várias flags, a ULA utiliza somente um bit auxiliar, denominado *Flag Out*, que é escrito no registrador DF. Esse bit, além de servir como flag (indicando, por exemplo, overflow em uma adição), é utilizado como bit auxiliar para operações sequenciais na ULA. Por exemplo, na subtração sucessiva de números de vários bytes, DF com valor 0 indica que existe um “vai um” para o próximo byte em cada passo. Dessa forma, se DF possui valor 0 após completar a subtração, sabemos que o resultado foi negativo e deve ser obtido pelo complemento de dois do valor em D. A entrada de *Carry* na ULA é feita a partir do *Data Flag*, que é justamente oriundo do registrador DF. Além disso, existe um bit de controle no

Datapath que, a partir de uma porta AND, define se o bit do DF será inserido ou não na ULA.

4 REGISTRADORES

O banco de registradores é um dos principais componentes da arquitetura do processador RCA 1802, sendo responsável pelo armazenamento temporário de dados, endereços, estados da CPU e informações utilizadas durante a execução das instruções. Neste projeto, o banco de registradores foi implementado como um subcircuito independente no Logisim Evolution, permitindo sua integração com os demais módulos da CPU de forma modular e organizada.

A implementação contempla os dezesseis registradores de uso geral (R0–R15), cada um com largura de 16 bits, além dos registradores auxiliares D, DF, P, X, N, I, T, IE e Q, conforme especificado para o processador RCA 1802. Cada registrador auxiliar desempenha uma função que colabora para o controle do sistema.

Os registradores R0–R15 constituem o banco principal de registradores do processador. Eles armazenam endereços e dados utilizados pelas instruções e podem exercer funções específicas de acordo com o contexto de execução. Por exemplo, o registrador selecionado por P atua como contador de programa (Program Counter), enquanto o registrador indicado por X é utilizado como registrador de índice para diversas operações de memória. Além disso, registradores como R0, R1 e R2 podem assumir funções especiais relacionadas ao DMA, interrupções e pilha, respectivamente.

O acesso ao banco de registradores foi dividido em operações independentes de leitura e escrita. Para a escrita foi utilizado um decoder 4→16, responsável por converter o sinal de seleção de escrita (Wsel) em dezesseis sinais individuais de habilitação (Write Enable), garantindo que apenas um registrador seja atualizado a cada ciclo de clock quando o sinal global de escrita estiver ativo.

A leitura dos registradores foi implementada por meio de um multiplexador 16→1, controlado pelo sinal interno Rsel. O valor presente no registrador selecionado é encaminhado para a saída de dados do banco, permitindo que outros módulos da

CPU, como a ULA e a interface de memória, utilizem seu conteúdo. Essa separação entre os caminhos de leitura e escrita possibilita que um registrador seja lido enquanto outro é escrito, tornando o banco mais flexível e compatível com a execução das microoperações definidas pela unidade de controle.

A seleção do registrador a ser lido não é realizada diretamente pela unidade de controle. Para isso foi implementado um multiplexador adicional responsável por gerar o sinal Rsel a partir dos registradores P, X e N. O registrador P identifica qual dos dezesseis registradores atua como contador de programa, sendo utilizado durante o ciclo de busca das instruções. O registrador X indica qual registrador será empregado como índice em operações de memória. Já o registrador N corresponde ao nibble menos significativo do opcode e identifica o registrador especificado diretamente pela instrução executada. A unidade de controle seleciona, por meio do sinal Rsel_source, qual dessas três fontes será utilizada para formar o valor de Rsel.

Além do banco principal, foram implementados os registradores auxiliares previstos na arquitetura do RCA 1802:

- D (8 bits): acumulador principal utilizado pela maioria das operações aritméticas e lógicas;
- DF (1 bit): flag de carry/borrow produzido pela ULA;
- P (4 bits): seleciona qual registrador R0–R15 será utilizado como Program Counter;
- X (4 bits): seleciona o registrador de índice utilizado nas operações indiretas;
- N (4 bits): armazena o nibble inferior do opcode, correspondente ao registrador referenciado pela instrução;
- I (4 bits): armazena o nibble superior do opcode, responsável por identificar a classe da instrução;
- T (8 bits): registrador temporário utilizado principalmente durante o tratamento de interrupções;
- IE (1 bit): habilita ou desabilita o sistema de interrupções do processador;
- Q (1 bit): flag de saída programável utilizada pelas instruções SEQ e REQ.

Cada registrador auxiliar foi implementado com entradas independentes de dados, sinal de habilitação de escrita (Write Enable), entrada de clock, entrada de reset e

saída de dados, permitindo que sejam controlados diretamente pela unidade de controle durante a execução das microoperações.

A implementação desenvolvida oferece suporte às instruções de manipulação de registradores previstas no projeto, incluindo GLO, GHI, PLO, PHI, INC, DEC, IRX, SEP e SEX. Nessas instruções, o banco de registradores interage diretamente com a unidade de controle, o decodificador de instruções e a ULA, desempenhando papel fundamental na execução correta das operações do processador.

A estrutura modular adotada facilita a integração do banco de registradores ao circuito principal da CPU. Todos os sinais de entrada e saída foram devidamente nomeados e organizados, permitindo que os demais módulos do projeto utilizem o banco de registradores sem necessidade de alterações internas. Essa abordagem torna a implementação mais legível, reutilizável e compatível com a arquitetura proposta para o microprocessador.

5 UNIDADE DE CONTROLE E μ ROM

A unidade de controle do CDP1802 foi implementada como uma unidade microprogramada, baseada em uma memória de controle (μ ROM) que armazena, para cada etapa de execução, os sinais que comandam a ULA, o banco de registradores, a memória, os flags e a interface de E/S. Essa abordagem evita o crescimento excessivo de uma lógica puramente combinacional diante das várias classes de instrução do CDP1802 — carga, armazenamento, aritmética, lógica, deslocamento, desvios curtos e longos, E/S e controle: cada instrução passa a ser traduzida em uma pequena sequência de microoperações, armazenadas como palavras binárias.

O controle está organizado em quatro blocos: o registrador de instrução (IR); a PLA/decodificador, que interpreta o opcode e gera a classe da instrução; a FSM, que indica a etapa atual da execução; e a própria μ ROM. A FSM tem três estados — FETCH, EXECUTE1 e EXECUTE2 —, codificados em 3 bits. FETCH busca o opcode e incrementa R(P); EXECUTE1 executa instruções simples ou a primeira

etapa das mais complexas; EXECUTE2 entra em ação quando é necessária uma etapa adicional, como em LDA ou nos desvios longos.

Cada posição da μ ROM guarda uma microinstrução de 32 bits, dividida em quinze campos: ALU_OP, SRC_A e SRC_B (operação e operandos da ULA), DST (destino do resultado), ADDR_SRC (origem do endereço de memória), REG_SEL_SRC (registrador selecionado), MEM_RD/MEM_WR, IR_LOAD, REG_INC/REG_DEC, DF_LOAD, Q_LOAD, COND_EN e NEXT_STATE.

O endereço da μ ROM é formado por $\mu\text{ADDR} = \{\text{STATE}[2:0], \text{INSTR_CLASS}[5:0]\}$. Como o nibble superior do opcode nem sempre identifica a instrução por completo — os opcodes F0–FF, por exemplo, compartilham esse nibble mas correspondem a operações diferentes, como LDX, OR, ADD e LDI —, a PLA recebe o opcode completo de 8 bits e gera uma classe expandida de 6 bits. Com 3 bits de estado e 6 de classe, a μ ROM tem 512 posições de 32 bits.

O circuito controle.circ, implementado no Logisim, reúne um registrador de estado de 3 bits (com reset síncrono para FETCH), um splitter que forma o endereço de 9 bits a partir do estado e da classe da instrução, o componente ROM correspondente à μ ROM (carregado a partir do arquivo uROM.mem) e um segundo splitter que decompõe a palavra lida da ROM nos quinze sinais de controle, disponibilizados como saídas do circuito para os demais módulos.

O arquivo uROM.mem já contém a codificação das 64 posições de FETCH, de 48 classes de instrução em EXECUTE1 e das 4 classes que dependem de EXECUTE2 (LDA, RET, DIS e desvio/skip longo). As demais posições permanecem reservadas, disponíveis para expansão futura, como interrupções ou DMA mais detalhados.

6 PLA / DECODIFICADOR

O decodificador (PLA) do 1802 recebe o opcode de 8 bits (4 bits mais significativos que definem a classe de instrução, o nibble I e os 4 menos significativos do que selecionam o registrador N) vindo do registrador de instrução e gera a saída INSTR_CLASS (6 bits), que é o campo utilizado pela uROM.

A tabela verdade criada para implementação do decodificador, contendo as definições de entrada do Opcode, nibble I, nibble N, Mnemônico e saída da

INSTR_CLASS, está no arquivo “TabelaDecoder.pdf” disponível no repositório do trabalho, assim como a imagem da ROM utilizada no circuito com o nome de “decoder_rom.mem”. Por fim, o circuito do decoder também está presente no repositório com o nome de “pla_decodificador.circ”, foram verificadas entradas e saídas específicas para fins de teste do circuito e os resultados obtidos estavam todos corretos.

7. MÓDULO DE MEMÓRIA E INTERFACE DE I/O

7.1 Barramento Multiplexado

Para reduzir o número de pinos físicos do encapsulamento do microprocessador CDP1802, a arquitetura utiliza a técnica de **multiplexação temporal** no barramento de endereços. Em vez de disponibilizar 16 linhas de endereço simultâneas, o processador envia os 16 bits divididos em duas metades de 8 bits através do mesmo barramento de estado de memória (MA0 a MA7).

O sincronismo e a captura dessas metades são controlados por dois sinais de clock especializados gerados pela CPU:

- **TPA (Timing Pulse A):** Disparado no início do ciclo de máquina, indica que a metade superior do endereço (Bits 8 a 15) está presente no barramento. O subcircuito `addr_latch` utiliza este pulso para armazenar o valor no registrador `ADDR_HIGH`.
- **TPB (Timing Pulse B):** Disparado na sequência, indica que a metade inferior do endereço (Bits 0 a 7) está disponível. O circuito captura este valor no registrador `ADDR_LOW`.

A combinação das saídas de ambos os registradores reconstrói o barramento de endereço completo de 16 bits (`ADDR_FULL`), garantindo a estabilidade do endereço durante todo o ciclo de leitura ou escrita na memória.

7.2 Interface de Memória

A memória principal do sistema é composta por uma memória **RAM volátil de 64K x 8 bits**. Esta configuração exige um barramento de endereço nativo de 16 bits

(permitindo o mapeamento de $2^{16} = 65.536$ posições de memória distintas) e um barramento de dados de 8 bits para leitura e escrita de bytes.

O controle de fluxo de dados na RAM é gerenciado por duas linhas de controle ativas:

- **MWR (Memory Write):** Quando em nível lógico alto (1) em sincronia com o clock do sistema, habilita a escrita do byte presente no barramento de entrada (**D_IN**) na posição de memória selecionada.
- **MRD (Memory Read):** Quando ativo, habilita a leitura da posição selecionada, jogando o byte armazenado para o barramento de saída (**D_OUT**).

Nota de Implementação (Prevenção de Conflito): Para evitar o fenômeno de conflito de barramento (*bus contention*), a saída da RAM é isolada por um **buffer tri-state**. Quando o sinal **MRD** está desativado (0), a saída entra em estado de alta impedância (alta resistência, indicado por **Hi-Z** ou **U**), desconectando eletricamente o componente do barramento comum e permitindo que outros periféricos ou o próprio processador escrevam na linha sem causar curto-circuito.

7.3 Controlador DMA (Direct Memory Access)

O controlador de DMA (**dma_ctrl**) permite a transferência de dados em alta velocidade entre dispositivos periféricos e a memória RAM sem a interceptação direta da CPU, otimizando o desempenho do sistema. Na arquitetura do CDP1802, o registrador interno **R0** funciona nativamente como o ponteiro de endereço para operações de DMA.

O bloco opera através de duas linhas de requisição externa:

1. **DMA_IN:** Solicita uma operação de leitura de um periférico para escrita direta na RAM.
2. **DMA_OUT:** Solicita uma operação de leitura da RAM para envio direto a um periférico.

Quando qualquer uma dessas requisições é ativada, o bloco **dma_ctrl** assume o controle das linhas de endereço através de multiplexadores internos, substituindo o

endereço do processador (**CPU_ADDR**) pelo endereço contido em **R0**. O controlador então:

- Sinaliza o sinal **DMA_ACK** para informar ao periférico que o barramento está disponível.
- Emite o pulso **R0_INC** direcionado ao bloco de registradores (Grupo 3), notificando-o para incrementar o valor de **R0** de forma automática ao fim do ciclo, preparando o ponteiro para o próximo byte da sequência.

7.4 Interface EF e Q

O módulo de Entrada/Saída (I/O) discreta gerencia a comunicação de flags de estado e saídas programáveis de 1 bit:

7.4.1 Flags Externas (EF1 a EF4)

São quatro linhas de entrada independentes (**EF1_IN** a **EF4_IN**) que permitem que dispositivos externos informem condições de status diretamente ao processador. Na arquitetura de software do CDP1802, essas linhas são mostradas diretamente por instruções de desvio condicional de curto escopo:

- **Instruções B1, B2, B3, B4:** Desviam o fluxo do programa se a respectiva flag estiver ativa (**1**).
- **Instruções BN1, BN2, BN3, BN4:** Desviam o fluxo do programa se a respectiva flag não estiver ativa (**0**).

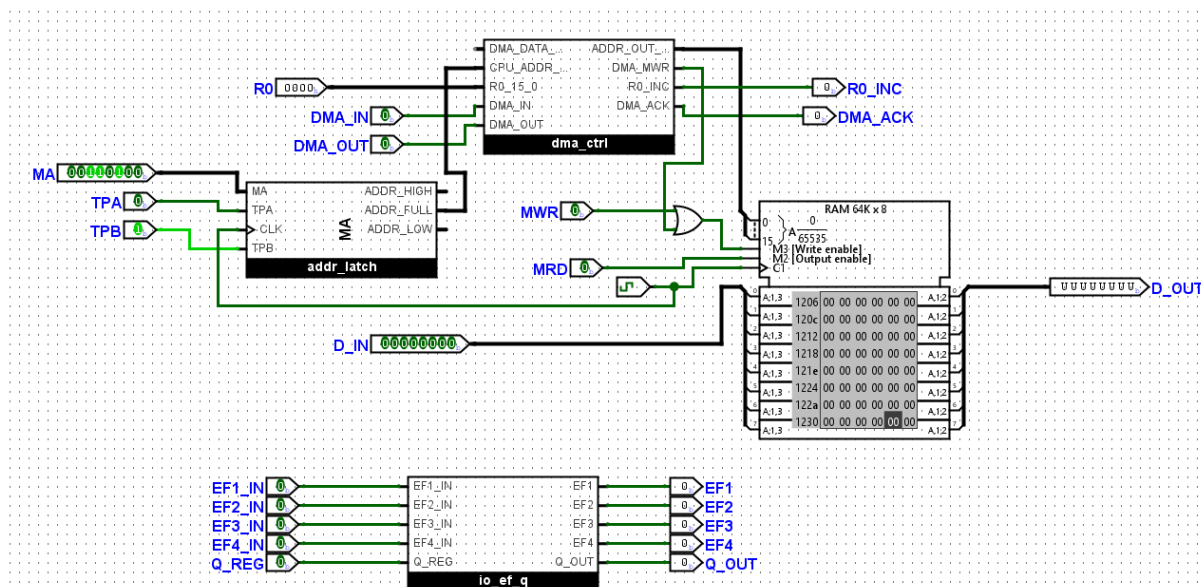
7.4.2 Saída Programável (Q)

O sinal **Q** é uma linha de saída controlada diretamente por um flip-flop interno da CPU, cujo estado é alterado via software através de duas instruções dedicadas:

- **SEQ (Set Q):** Define a saída **Q** para o nível lógico alto (**1**).
- **REQ (Reset Q):** Limpa a saída **Q** voltando-a para o nível lógico baixo (**0**).
Essa linha é comumente utilizada para gerar sinais de sincronismo externos, comunicação serial rudimentar ou controle de LEDs de diagnóstico.

7.5 Diagrama do Módulo

Abaixo apresenta-se a captura de tela do circuito integrado e finalizado no ambiente Logisim, evidenciando as conexões de interligação entre os subcircuitos descritos anteriormente (`addr_latch`, `dma_ctrl`, a memória RAM 64K e as portas de E/S).



8. TESTES EM ASSEMBLY

8.1 Visão Geral dos Programas de Teste

Para verificar o funcionamento correto dos módulos implementados, foram desenvolvidos oito programas de teste em assembly. Cada programa foi escrito para exercitar um conjunto específico de instruções e, ao final da execução, o estado do acumulador D, da flag DF e dos registradores internos reflete os resultados esperados, possibilitando a validação comparativa no simulador Logisim. Os arquivos gerados no formato .mem são carregados diretamente no componente RAM de 64K x 8 bits do circuito `memoria_io` para execução.

8.2 test_regs

O programa `test_regs` valida o conjunto de instruções validado pela leitura, escrita e seleção dos registradores internos de 16 bits. As instruções cobertas são: GLO, GHI, PLO, PHI, INC, DEC, IRX, SEP e SEX. A execução pode ser dividida em três etapas:

- **Escrita e leitura do R5:** o registrador R5 é inicializado com o valor AA55H. Em seguida, GLO R5 copia o byte baixo (55H) para D e GHI 5R copia o byte alto (AAH). INC R5 incrementa R5 para AA56H. Duas execuções consecutivas de DEC R5 reduzem ele para AA54H;
- **Incremento via IRX:** o registrador R6 é carregado com 0040H e selecionado como registrador de índice com SEX R6. A instrução IRX incrementa R6 para 0041H sem alterar D;
- **Troca de PC com SEP:** R7 é carregado com 001EH. SEP R7 transfere o controle da CPU para o endereço 001EH, onde LDI 42H é executado. Em seguida, SEP R0 devolve o controle ao registrador R0.

8.3 test_alu_arith

O programa test_alu_arith verifica as operações aritméticas da ALU. As instruções cobertas são: ADI, ADCI, ADD, ADC, SMI, SMI, SM, SMB, SDI, SDBI, SD e SDB. O registrador R2 é iniciado em 0020H como ponteiro base para acesso à RAM.

A sequência de verificações é executada da seguinte forma:

- **Adição imediata com carry:** LDI 80H / ADI 90H resulta em D = 10H com DF = 1 (carry gerado). ADCI 02H computa $D = 05H + 02H + DF(1) = 08H$, com DF = 0;
- **Adição com operando em memória:** M(0020H) é preenchido com FFH via STR R2. ADD calcula $D = 01H + FFH = 00H$ (DF = 1). ADC computa $D = 00H + M(RX) + DF(1) = 00H$, com DF = 1;
- **Subtração imediata:** SMI 03H subtrai 03H de D = 05H, obtendo 02H com DF = 1 (sem borrow). SMI 05H produz FDH com DF = 0 (com borrow). SMI 03H computa $D = 0AH - 03H - NOT(DF = 0) = 06H$, com DF = 1;
- **Subtração reversa imediata:** SDI 05 computa $D = 05H - 02H = 03H$ (DF = 1). SDBI 04H calcula $D = 04H - 04H - NOT(DF = 1) = 00H$, com DF = 1;
- **Subtração com operando em memória:** SEX R2 configura X = 2 e M(0020H) é atualizado para 10H. SM calcula $D = 15H - 10H = 05H$ (DF = 1). SMB obtém $D = 05H - 10H - NOT(DF = 1) = F5H$ (DF = 0). SD computa $D = 10H - 20H = F0H$ (DF = 0). SBD calcula $D = 10H - F0H - NOT(DF = 0) = 1FH$ (DF = 0).

8.4 test_alu_logic

O programa `test_alu_logic` valida as seis operações lógicas da ALU nas modalidades imediata e com operando em memória. As instruções cobertas são: ORI, ANI, XRI, OR, AND e XOR. O registrador R4 é iniciado com 0030H e selecionado como registrador de índice com `SEX R4`. Para estabelecer `DF = 1`, `LDI 80H` e `ADI 80H` são executados, o que gera overflow e resulta `D = 00H`, com `DF = 1`.

A sequência de operações imediatas é:

- **ORI 55H:** $D = 00H \text{ OR } 55H = 55H$;
- **ANI 0FH:** $D = 55H \text{ AND } 0FH = 05H$;
- **XRI 35H:** $D = 05H \text{ XOR } 35H = 30H$.

Em seguida, `M(0030H)` é carregado com `AAH` via `STR R4`, e as operações com operando em memória são realizadas:

- **OR:** $D = 55H \text{ OR } AAH = FFH$;
- **AND:** $D = FFH \text{ AND } AAH = AAH$;
- **XOR:** $D = AAH \text{ XOR } AAH = 00H$.

8.5 test_shift

O programa `test_shift` verifica as instruções de deslocamento. Logo, as instruções cobertas são: SHR, SHRC, SHL e SHLC. Em todos os casos, o bit deslocado para fora do byte é capturado pela flag `DF`.

- **SHR:** `LDI 01H` e `SHR` deslocam o bit 0 para `DF`, resultando em $D = 00H$ e `DF = 1`;
- **SHRC:** `LDI 80H` e `SHRC` inserem `DF = 1` no bit 7, produzindo $D = C0H$ e `DF = 0`. Um segundo `SHRC` insere `DF = 0` no bit 7, resultando em $D = 60H$ e `DF = 0`;
- **SHL:** `LDI 80H` e deslocam o bit 7 para `DF`, resultando em $D = 00H$ e `DF = 1`;
- **SHCL:** `LDI 01H` e `SHCL` inserem `DF = 1` no bit 0, produzindo $D = 03H$ e `DF = 0`.

8.6 test_load_store

O programa `test_load_store` exercita os modos de acesso à memória. Logo, as instruções cobertas são: LDI, LDN, LDA, STR, STXD, LDXA e LDX. O registrador R3 é iniciado com 1500H, servindo de ponteiro base para todas as operações.

- **STR R3 e LDN R3:** LDI 55H e STR R3 gravam 55H em M(1500H). LDN R3 carrega, em D, M(R3) sem modificar R3, obtendo D = 55H;
- **LDA R3:** D = M(R3) = 55H e R3 é pós-incrementado para 1501H;
- **STXD:** SEX R3 define X = 3. LDI AAH e STXD gravam AAH em M(R3 = 1501H) e decrementam R3 para 1500H;
- **LDXA:** D = M(R3 = 1500H) = 55H e R3 é pós-incrementado para 1501H;
- **LDX:** D = M(R3 = 1501H) = AAH e R3 permanece em 1501H.

8.7 test_branch

O programa `test_branch` valida instruções de desvio de fluxo e de salto. Logo, as instruções cobertas são: BR, BZ, BNZ, BDF, BQ, BNQ, LBR, LBNZ, SKP e LSKP. O estado inicial é preparado com REQ (Q = 0), LDI 01H e SHR (D = 00H e DF = 1).

- **BR:** BR 07H desvia para o endereço 0007H;
- **BNF e BDF:** BNF 2EH não desvia, já que DF = 1. BDF 0DH desvia para 000DH;
- **BNZ e BZ:** BNZ 2EH não desvia, já que D = 00H. BZ 13H desvia para 0013H;
- **BNQ e BQ:** BNQ 17H desvia para 0017H, já que Q = 0. SEQ define Q = 1 e BQ 1CH desvia para 001CH;
- **SKP e LSKP:** SKP pula a instrução seguinte (LDI 55H). LSKP pula os 2 bytes seguintes (LDI AAH e operando);
- **LBR e LBNZ:** LBR 0025H realiza um salto incondicional longo. LBNZ desvia condicionalmente com D = 01H, alcançando o endereço em que LDI 99H é executado.

8.8 test_io

O programa `test_io` verifica a interface de I/O. Logo, cobre as instruções: SEQ, REQ, B1, B2, B3, B4, OUT e IN. As flags EF1, EF2, EF3 e EF4 são testadas também. O registrador R8 é iniciado com 0050H e definido como registrador de índice com SEX R8.

- **SEQ e REQ:** SEQ define $Q = 1$ e REQ retorna $Q = 0$ em seguida;
- **B1 até B4:** B1 0DH, B2 10H, B3 13H e B4 16H são executadas em sequência, cada uma seguida de SKP. Se a flag EF correspondente estiver ativa, o desvio é tomado. Caso contrário, SKP avança para a próxima verificação;
- **OUT 1 e IN 1:** M(0050H) é preenchido com A5H via STR R8. OUT 1 envia o conteúdo de M(R8) ao barramento de saída e pós-incrementa R8 para 0051H. DEC R8 decrementa R8 para 0050H. IN 1 captura o valor do barramento de entrada em D e grava ele em M(R8).

8.9 test_cypmem

O programa test_cypmem implementa uma rotina completa de cópia de bloco de memória. Por isso, cobre as instruções: LDI, PHI, PLO, STR, INC, LDA, DEC, GLO e BNZ.

A inicialização do programa se dá da seguinte forma:

- **R9 = 0060H** (ponteiro de origem);
- **RA = 0070H** (ponteiro de destino);
- **RC = 0004H** (contador de bytes);
- **M(0060H até 0063H) = DEH, ADH, BEH e EFH** (bloco de dados de origem, gravado via STR + INC sobre R2).

O laço de cópia é executado quatro vezes com a seguinte sequência de instruções:

- **LDA R9:** carrega M(R9) em D e pós-incrementa R9;
- **STR RA:** grava o valor de D em M(RA);
- **INC RA:** incrementa o valor de destino;
- **DEC RC, GLO RC e BNZ 24H:** decrementa o contador RC, copia o byte baixado de RC para D e desvia de volta ao início do laço enquanto D for diferente de 0.

Ao terminar as quatro iterações, RC.L = 00H e BNZ não desvia, o que encerra o laço.

9 RESPONSABILIDADES

Grupo	Aluno(s)
Instruções	Enrico Massariol Caliar, Guido Agnezi Deniz
ULA	Dimitry Deveza, Enrico Polez Ferreira Pinto
Registradores	Eduardo Vitor Ogawa
Unidade de Controle + μ ROM	Djan Carlos de Moura Ferreira Junior, Gabriel Carminati, George Cirilo do Nascimento
PLA / Decodificador	David Maquarte Gumes e Silva
Módulo de Memória e Interface de I/O	Guilherme Cardoso Delatorre
Testes em Assembly	Enzo Venturini Baptista
Integração + Relatório	Todos

REFERÊNCIAS

INTERSIL CORPORATION. CDP1802A/AC/BC: CMOS 8-bit microprocessors. [S.l.]: Intersil, 1999. Disponível em:
<http://www.cosmacelf.com/publications/data-sheets/cdp1802.pdf>. Acesso em: 22 jun. 2026.